

INDEX

S.N	Name of the experiment	Page.no	Date	Signature
1.	Create Comma Separate Files (CSV), Load CSV files into internal Data Structure.			
2.	Python Programming for Loading the Data.			
3.	Python Programming for Cleaning the Data.			
4.	Python Programming for Visualization.			
5.	Python library Scikit-learn to handle regression with python.			
6.	Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select the appropriate data set for your experiment and draw graphs.			
7	Implement a Simple linear regression model using python to predict a response using a single feature.			
8	Implement a Multiple linear regression model to show the relationship between two or more features and a response by fitting a linear equation to observed data.			
9	Implement a regression model to predict house prices.			
10.	Implement and demonstrate the FIND-S (EM algorithm) algorithm for finding the most specific hypothesis based on a given set of training data samples. Read the training data from a .CSV file.			

EXPERIMENT NO.: 1		
Title of Experiment: Create Comma Separate Files (CSV), Load CSV files into internal Data Structure.		
Program: B.Tech CSE	Course: Machine Learning LAB Course Code: PCS 311(L)	Semester: V
Page no: ¼		

Create and Save CSV File

```
import csv
```

```
# Sample data
```

```
data = [
    ["ID", "Name", "Marks"],
    [1, "Saurabh", 85],
    [2, "Bob", 78],
    [3, "Charlie", 92],
    [4, "David", 67]
]
```

```
# Create and save CSV file in Colab environment
```

```
with open("students.csv", mode="w", newline="") as file:
```

```
    writer = csv.writer(file)
```

```
    writer.writerows(data)
```

```
print("CSV file 'students.csv' created successfully!")
```

Output -

```
CSV file 'students.csv' created successfully!
```

```
students.csv X
Python > machine_learning > p
1 ID,Name,Marks
2 1,Sarvesh,85
3 2,Bob,78
4 3,Charlie,92
5 4,David,67
```

a) Load as List of Lists

```
data_list = []
```

```
with open("students.csv", mode="r") as file:
```

```
    reader = csv.reader(file)
```

```
    for row in reader:
```

```
        data_list.append(row)
```

```
print("🔍 Loaded as List of Lists:")
```

```
print(data_list)
```

Output -

```
🔍 Loaded as List of Lists:
[['ID', 'Name', 'Marks'], ['1', 'Saurabh',
'85'], ['2', 'Bob', '78'], ['3', 'Charlie',
'92'], ['4', 'David', '67']]
```

b) Load as List of Dictionaries

```
data_dict_list = []

with open("students.csv", mode="r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        data_dict_list.append(dict(row))

print("🔍 Loaded as List of Dictionaries:")
print(data_dict_list)
```

Output -

```
Loaded as List of Dictionaries:
[{'ID': '1', 'Name': 'Saurabh', 'Marks': '85'},
 {'ID': '2', 'Name': 'Bob', 'Marks': '78'}, {'ID':
 : '3', 'Name': 'Charlie', 'Marks': '92'}, {'ID':
 '4', 'Name': 'David', 'Marks': '67'}]
```

EXPERIMENT NO.: 2		
Title of Experiment: Python Programming for Loading the Data.		
Program: B.Tech CSE	Course: Machine Learning LAB Course Code: PCS 311(L)	Semester: V
Page no: ¼		

Create and Save CSV File (if not already created)

```
import csv
```

```
# Sample data
```

```
data = [
    ["ID", "Name", "Marks"],
    [1, "Saurabh", 85],
    [2, "Bob", 78],
    [3, "Charlie", 92],
    [4, "David", 67]
]
```

```
# Write to CSV
```

```
with open("students.csv", mode="w", newline="") as file:
```

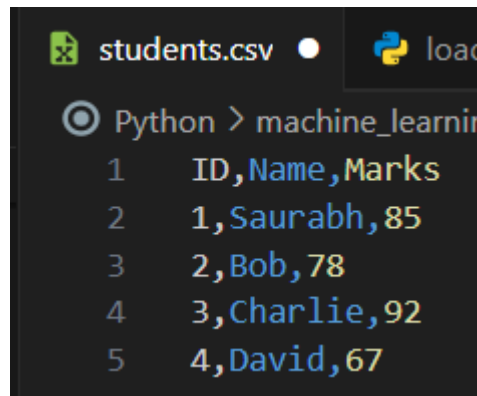
```
    writer = csv.writer(file)
```

```
    writer.writerows(data)
```

```
print("CSV file created.")
```

Output -

CSV file created.



```
students.csv load
Python > machine_learning
1 ID,Name,Marks
2 1,Saurabh,85
3 2,Bob,78
4 3,Charlie,92
5 4,David,67
```

Load CSV into List of Dictionaries

```
data_dict = []
```

```
with open("students.csv", mode="r") as file:
```

```
    reader = csv.DictReader(file)
```

```
    for row in reader:
```

```
        data_dict.append(row)
```

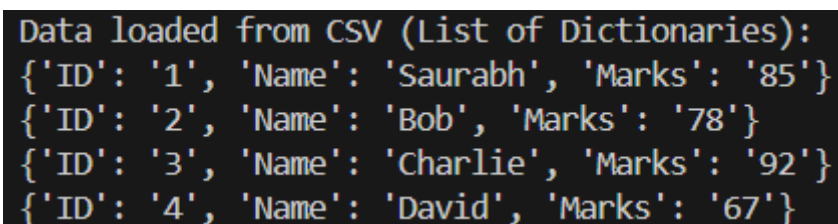
```
# Display loaded data
```

```
print("Data loaded from CSV (List of Dictionaries):")
```

```
for item in data_dict:
```

```
    print(item)
```

Output -



```
Data loaded from CSV (List of Dictionaries):
{'ID': '1', 'Name': 'Saurabh', 'Marks': '85'}
{'ID': '2', 'Name': 'Bob', 'Marks': '78'}
{'ID': '3', 'Name': 'Charlie', 'Marks': '92'}
{'ID': '4', 'Name': 'David', 'Marks': '67'}
```

Import pandas and Load CSV

```
import pandas as pd

# Load the CSV file into a DataFrame
df = pd.read_csv("students.csv")

# Display the data
print("Data loaded using pandas:")
print(df)
```

Access Individual Columns or Rows

```
# Access 'Name' column
print("\nStudent Names:")
print(df["Name"])

# Access specific row
print("\nSecond Row:")
print(df.iloc[1]) # Index starts from 0
```

Output -

Data loaded using pandas:

	ID	Name	Marks
0	1	Saurabh	85
1	2	Bob	78
2	3	Charlie	92
3	4	David	67

Student Names:

0	Saurabh
1	Bob
2	Charlie
3	David

Name: Name, dtype: object

Second Row:

ID	2
Name	Bob
Marks	78

Name: 1, dtype: object

EXPERIMENT NO.: 3		
Title of Experiment: Python Programming for Cleaning the Data.		
Program: B.Tech CSE	Course: Machine Learning LAB Course Code: PCS 311(L)	Semester: V
Page no: ¼		

Step-by-Step: Data Cleaning with Python

1. Import Required Library

```
import pandas as pd
```

2. Load the CSV File

```
# Load CSV data into a DataFrame
```

```
df = pd.read_csv("students.csv")
```

```
print("Original Data:")
```

```
print(df)
```

Output-

```
Original Data:
   ID  Name  Marks
0   1  Saurabh  85.0
1   2    Bob   NaN
2   3  Charlie  92.0
3   4   David  67.0
```

3. Common Data Cleaning Tasks

a) Check for Missing Data

```
print("Missing values in each column:")
```

```
print(df.isnull().sum())
```

Output -

```
Missing values in each column:
ID      0
Name    0
Marks   1
dtype: int64
```

b) Fill Missing Values (Imputation)

```
# Fill missing Marks with the average
```

```
df['Marks'] = df['Marks'].fillna(df['Marks'].mean())
```

Output -

```
0    85.000000
1    81.333333
2    92.000000
3    67.000000
Name: Marks, dtype: float64
```

```
# OR: Fill missing Name with "Unknown"
```

```
df['Name'] = df['Name'].fillna("Unknown")
```

c) Remove Rows with Missing Values

```
# Drop rows with any missing values
```

```
df_cleaned = df.dropna()
```

```
# OR: Drop rows with missing 'Marks' only
```

```
# df_cleaned = df.dropna(subset=['Marks'])
```

Output -

	ID	Name	Marks
0	1	Saurabh	85.0
2	3	Charlie	92.0
3	4	David	67.0

d) Remove Duplicate Records

```
df = df.drop_duplicates()
```

e) Convert Data Types

```
# df['Marks'] = df['Marks'].astype(int)
```

```
df['ID'] = df['ID'].astype(int)
```

Output -

```
0    1.0
1    2.0
2    3.0
3    4.0
Name: ID, dtype: float64
```

f) Strip Whitespace and Clean Strings

```
df['Name'] = df['Name'].str.strip().str.title()
```

g) Filter Invalid Data (e.g., negative marks)

```
df = df[df['Marks'] >= 0]
```

4. Save Cleaned Data

```
df.to_csv("students_cleaned.csv", index=False)  
print("Cleaned data saved as 'students_cleaned.csv'")
```

Output -

```
Cleaned data saved as 'students_cleaned.csv'
```

EXPERIMENT NO.: 4		
Title of Experiment: Python Programming for Visualization.		
Program: B.Tech CSE	Course: Machine Learning LAB Course Code: PCS 311(L)	Semester: V
Page no: 1/4		

Step-by-Step: Data Visualization in Python

We'll use:

- `matplotlib` – for basic 2D plots
- `seaborn` – for statistical visualizations
- `pandas` – for data handling

1. Import Required Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

2. Sample Data

```
data = {
    'Name': ['Saurabh', 'Bob', 'Charlie', 'David', 'Eva'],
    'Marks': [85, 78, 92, 67, 88],
    'Subject': ['Math', 'Math', 'Math', 'Math', 'Math']
}

df = pd.DataFrame(data)
print(df)
```

Output -

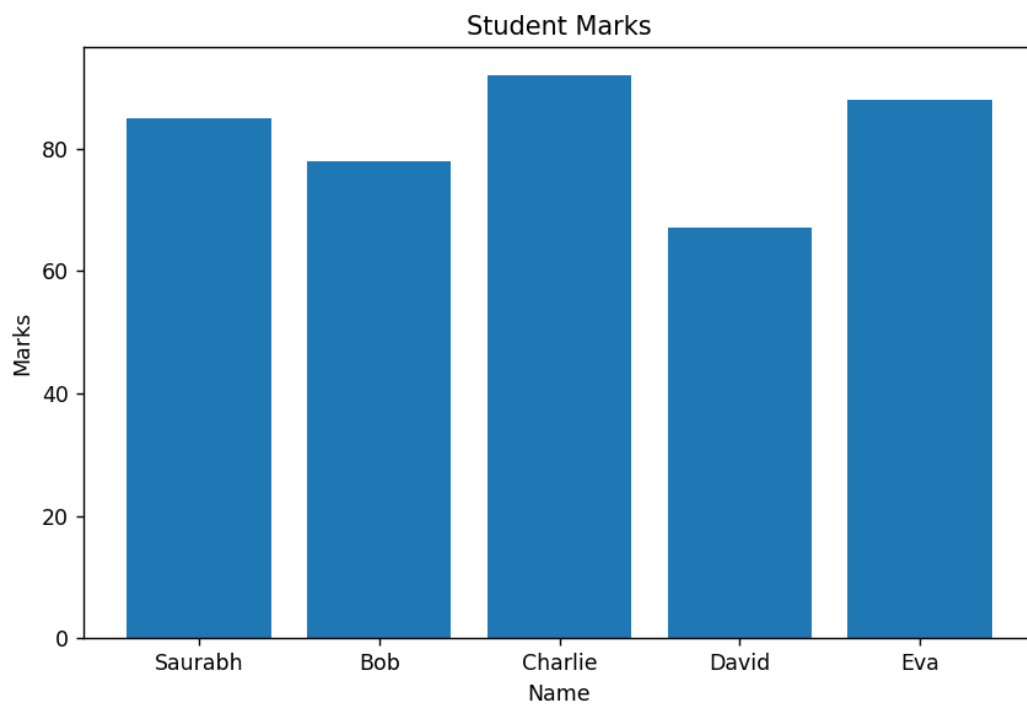
	Name	Marks	Subject
0	Saurabh	85	Machine Learning
1	Bob	78	DOTnet
2	Charlie	92	DSA
3	David	67	Computer Networks
4	Eva	88	Fullstack

3. Basic Visualizations

a) Bar Chart (Name vs Marks)

```
plt.figure(figsize=(8, 5))  
plt.bar(df['Name'], df['Marks'])  
plt.title('Student Marks')  
plt.xlabel('Name')  
plt.ylabel('Marks')  
plt.show()
```

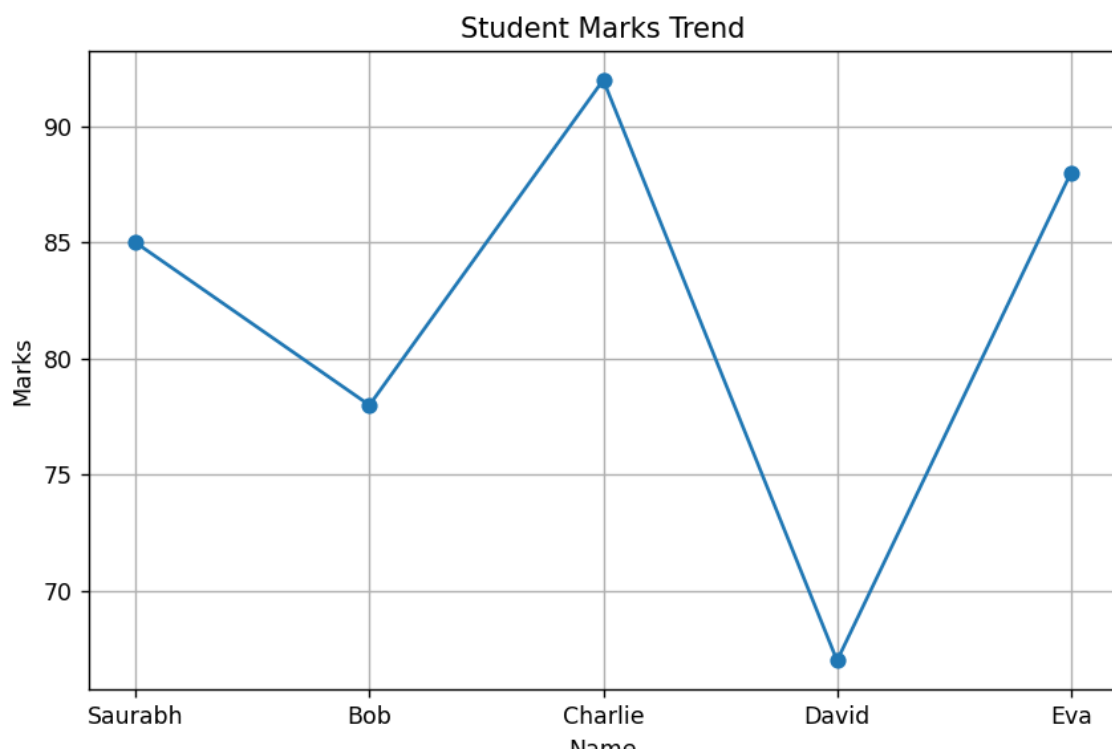
Output -



b) Line Chart

```
plt.figure(figsize=(8, 5))
plt.plot(df['Name'], df['Marks'], marker='o')
plt.title('Student Marks Trend')
plt.xlabel('Name')
plt.ylabel('Marks')
plt.grid(True)
plt.show()
```

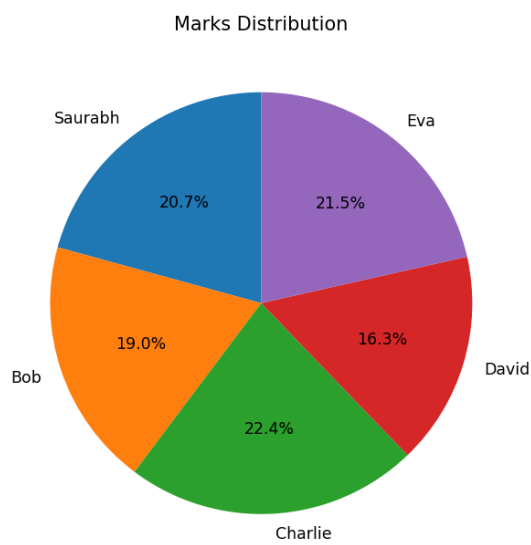
Output -



c) **Pie Chart**

```
plt.figure(figsize=(6, 6))
plt.pie(df['Marks'], labels=df['Name'], autopct='%1.1f%%', startangle=90)
plt.title('Marks Distribution')
plt.show()
```

Output -

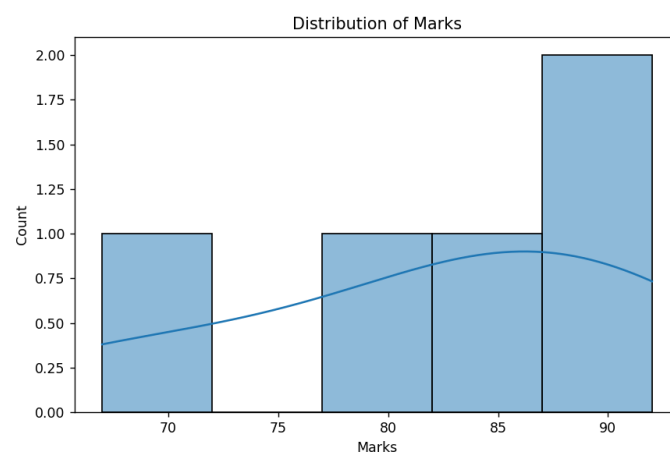


4. Statistical Visualizations using seaborn

a) Histogram

```
plt.figure(figsize=(8, 5))  
sns.histplot(df['Marks'], bins=5, kde=True)  
plt.title('Distribution of Marks')  
plt.show()
```

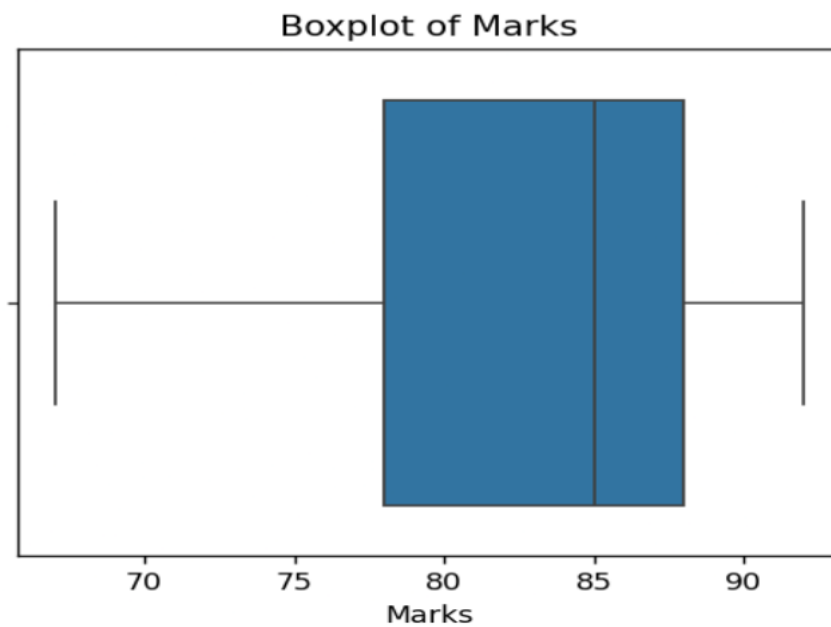
Output -



b) Box Plot (to detect outliers)


```
plt.figure(figsize=(6, 4))
sns.boxplot(x=df['Marks'])
plt.title('Boxplot of Marks')
plt.show()
```

Output -

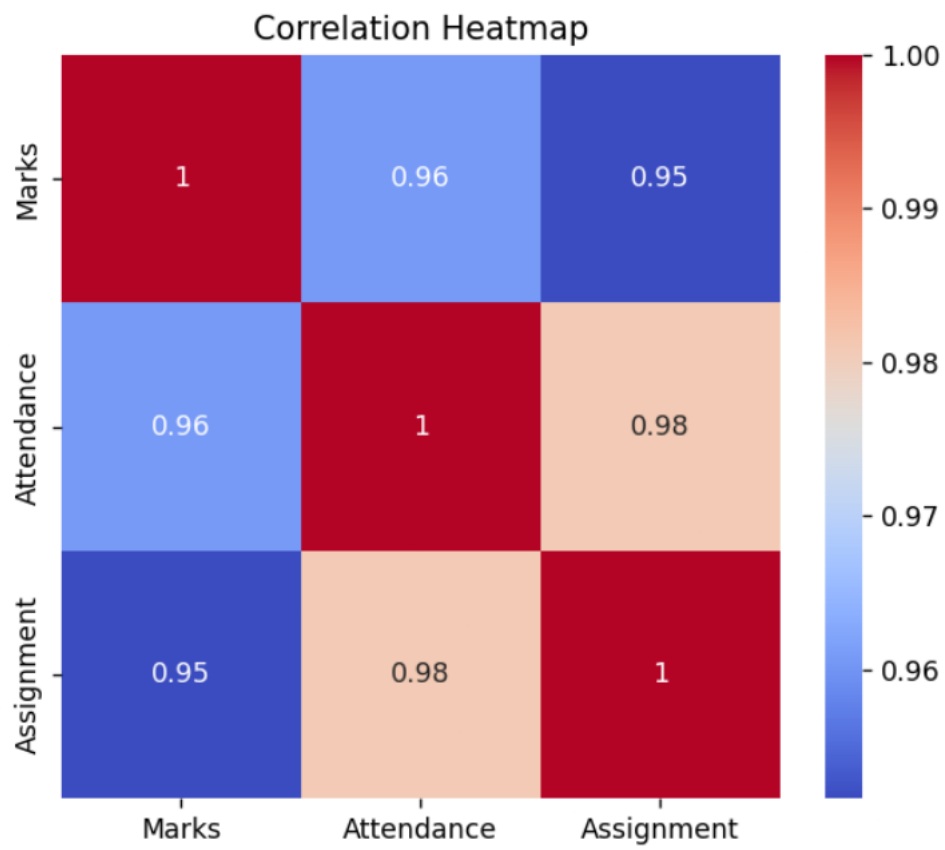


c) Heatmap (if you have a correlation matrix)

```
# Add extra numeric columns for heatmap demo
df['Attendance'] = [90, 80, 95, 60, 85]
df['Assignment'] = [88, 70, 90, 55, 80]

plt.figure(figsize=(6, 5))
sns.heatmap(df[['Marks', 'Attendance', 'Assignment']].corr(), annot=True,
            cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

Output -



EXPERIMENT NO.: 5		
Title of Experiment: Python library Scikit-learn to handle regression with python.		
Program: B.Tech CSE	Course: Machine Learning LAB Course Code: PCS 311(L)	Semester: V
Page no: 1/4		

Scikit-learn is a powerful Python library for machine learning. It supports supervised and unsupervised learning and provides tools for model fitting, data preprocessing, evaluation, and more.

Step-by-Step: Linear Regression with Scikit-learn

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
```

Step 2: Prepare the Dataset

```
# Sample data
data = {
    'Hours': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Marks': [20, 35, 40, 50, 60, 65, 70, 80, 85, 95]
}

df = pd.DataFrame(data)
```

Print(df)

Output-

	Hours	Marks
0	1	20
1	2	35
2	3	40
3	4	50
4	5	60
5	6	65
6	7	70
7	8	80
8	9	85
9	10	95

Step 3: Split Features and Target

```
X = df[['Hours']] # Features (2D)
```

```
y = df['Marks'] # Target
```

Step 4: Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Step 5: Create and Train the Model

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

Step 6: Make Predictions

```
y_pred = model.predict(X_test)
```

```
# Show predictions vs actual
```

```
for i in range(len(X_test)):
```

```
print(f"Study Hours: {X_test.iloc[i,0]} → Actual Marks: {y_test.iloc[i]}, Predicted: {y_pred[i]:.2f}")
```

Output -

```
Study Hours: 9 → Actual Marks: 85, Predicted: 88.36
Study Hours: 2 → Actual Marks: 35, Predicted: 31.64
```

Step 7: Visualize Regression Line

Plotting training data

```
plt.scatter(X, y, color='blue', label='Actual Data')
```

Regression line

```
plt.plot(X, model.predict(X), color='red', label='Regression Line')
```

```
plt.xlabel('Hours Studied')
```

```
plt.ylabel('Marks Obtained')
```

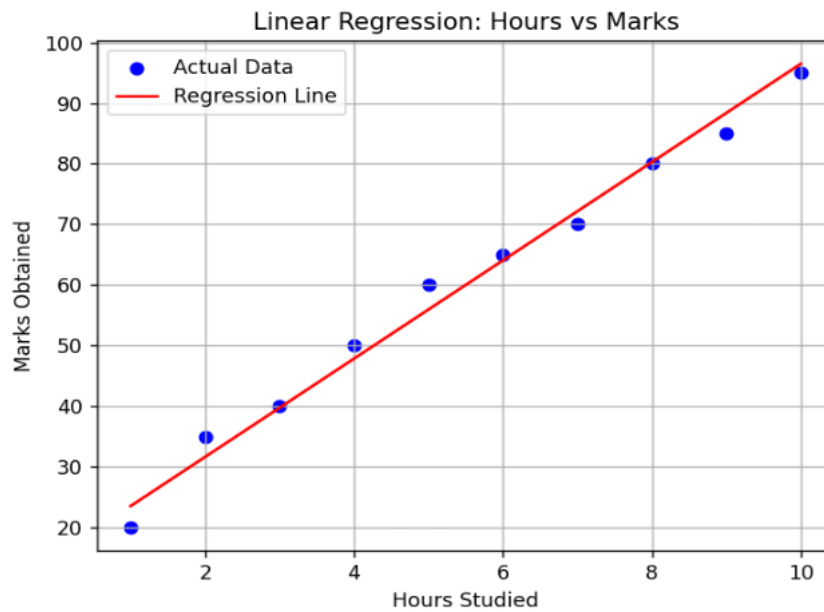
```
plt.title('Linear Regression: Hours vs Marks')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

Output -



Step 8: Evaluate the Model

```
mse = mean_squared_error(y_test, y_pred)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print(f"Mean Squared Error: {mse:.2f}")
```

```
print(f"R2 Score (Accuracy): {r2:.2f}")
```

Output -

```
Mean Squared Error: 11.30  
R2 Score (Accuracy): 0.98
```

EXPERIMENT NO.: 6		
Title of Experiment: Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select the appropriate data set for your experiment and draw graphs.		
Program: B.Tech CSE	Course: Machine Learning LAB Course Code: PCS 311(L)	Semester: V
Page no: ¼		

What is Locally Weighted Regression (LWR)?

- LWR is a **non-parametric regression algorithm**.
- It **fits a line to nearby points** instead of the whole dataset, assigning **weights using a Gaussian kernel**.
- Ideal for fitting **non-linear curves**.

Step-by-Step Implementation of LWR in Python

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Sample Data (e.g., quadratic)
X = np.linspace(1, 10, 100)
Y = np.sin(X) + np.random.randn(100) * 0.1 # Noisy sin wave

# Convert to matrix format for linear algebra
X = X[:, np.newaxis] # shape (100, 1)
X_b = np.hstack((np.ones_like(X), X)) # Add bias term → shape (100, 2)
m = len(X)
```

Step 2: Define the Weight Function

```
def get_weights(X_query, X_train, tau):
    """Compute the weights using the Gaussian kernel."""
    m = X_train.shape[0]
```

```

weights = np.eye(m)
for i in range(m):
    diff = X_train[i] - X_query
    weights[i, i] = np.exp(-(diff @ diff.T) / (2 * tau ** 2))
return weights

```

Step 3: Define LWR Prediction Function

```

def locally_weighted_regression(X_train, y_train, X_test, tau=0.5):
    """Predict using LWR for each test point."""
    m = X_test.shape[0]
    y_pred = np.zeros(m)

    for i in range(m):
        W = get_weights(X_test[i], X_train, tau)
        #  $\theta = (X^T W X)^{-1} X^T W y$ 
        XTWX = X_train.T @ W @ X_train
        XTWy = X_train.T @ W @ y_train
        theta = np.linalg.pinv(XTWX) @ XTWy
        y_pred[i] = X_test[i] @ theta
    return y_pred

```

Step 4: Fit and Visualize the Model

```

# Fit LWR
tau = 0.5 # Bandwidth parameter
y_pred = locally_weighted_regression(X_b, Y, X_b, tau)

# Plotting

```



```

plt.figure(figsize=(10, 6))
plt.scatter(X[:, 0], Y, label='Training Data', color='lightgray')
plt.plot(X[:, 0], y_pred, color='red', label='LWR Prediction')
plt.title(f'Locally Weighted Regression (tau = {tau})')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.grid(True)
plt.show()

```

Output -

